3D-Printed AI-Powered Microscope and Cellular Imaging Studio

User Manual

Developers

Addison "Danny" Drobny, Dylan Lee, Kirby Burke, Mina Li, and Roberto Lucatero

Affiliation

UC Irvine Departments of Information & Computer Science (ICS) and Electrical Engineering and Computer Science (EECS)

Table of Contents

Introduction

For our Senior Capstone Project, we fabricated a 3D-printed microscope and developed the *Cellular Imaging Studio* program – based on the [OpenFlexure Project] – to provide a low-cost solution for students, educators, and research professionals to capture high resolution cellular imagery, and leverage the added features of cell counting and motion tracking, with robust reporting.

Features

In addition to the basic OpenFlexure microscope's basic capabilities for capturing and annotating images and adjusting the microscope image quality, we added the following features via the *Cellular Imaging Studio* program.

- Video capture
- Cell detection and counting from captured images
- Cell counting and motion tracking from video images
- Exporting count and motion insights as CSV files

Requirements

The following resources are required or recommended to build and use the microscope and Cellular Imaging Studio program.

Hardware Requirements

- Assembled OpenFlexure High-Resolution Microscope
    - Achromatic lens (optional)
    - 40X plan objective lens (optional)
- Windows 10 or 11 PC
- 4+ GB of RAM
- 1+ GHz processor
- Ethernet (i.e., RJ45) port or adapter
- Ethernet cable

Software Requirements

- Software Libraries (for Build):
    - npm
    - FFmpeg
    - Node.js
- Disk Space:
    - 203 MB for *Cellular Imaging Studio* program
    - 1 GB recommended for captured media

The build process includes the installation of additional software listed in the Appendix.

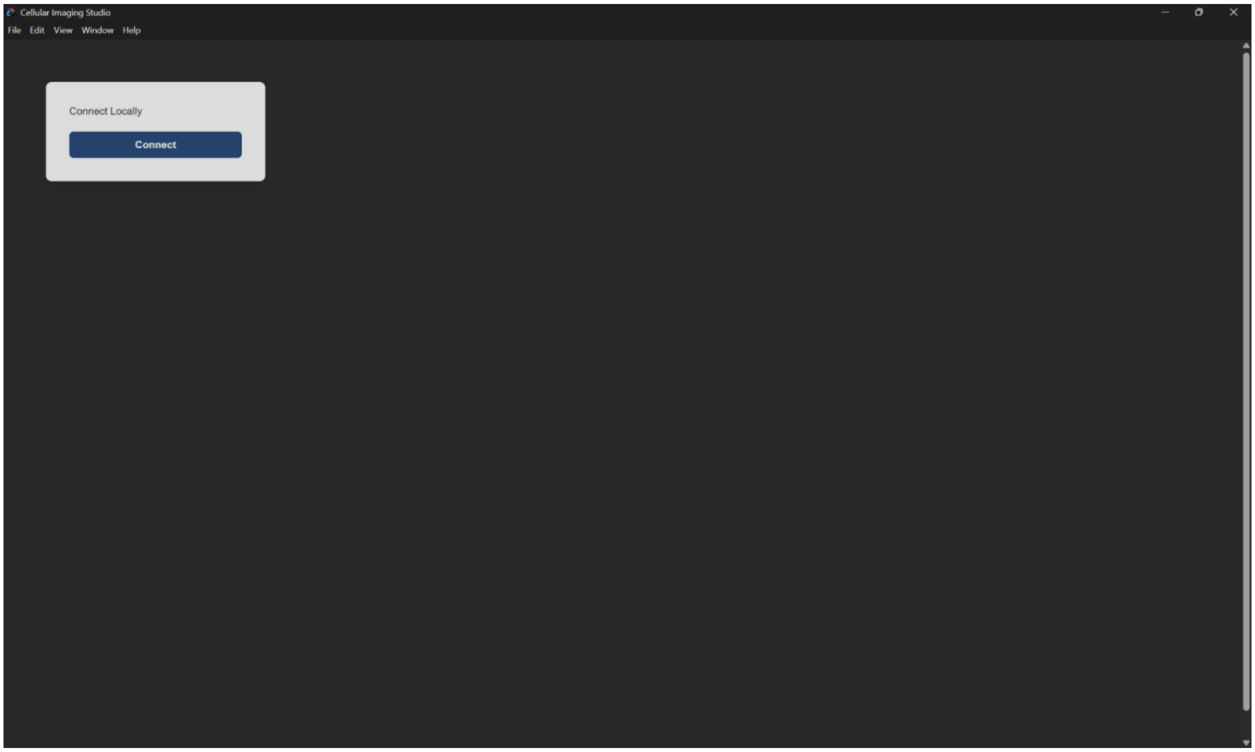
Installation

For basic installation, we recommend downloading and running our standalone installer: [Cellular Imaging Studio Setup 1.0.0.exe]

For those who want to build the installer themselves, we recommend cloning our GitHub repo and following the installation instructions at https://github.com/kirby-eecs26/ml-microscope.

Post-Installation

Prior to connecting to the microscope from the *Cellular Imaging Studio* program, plug the microscope into a stable power source and connect it to your computer with an Ethernet cable. The microscope may take several minutes to power up. So, please be patient. The microscope's port LEDs will blink green when it's ready to connect.



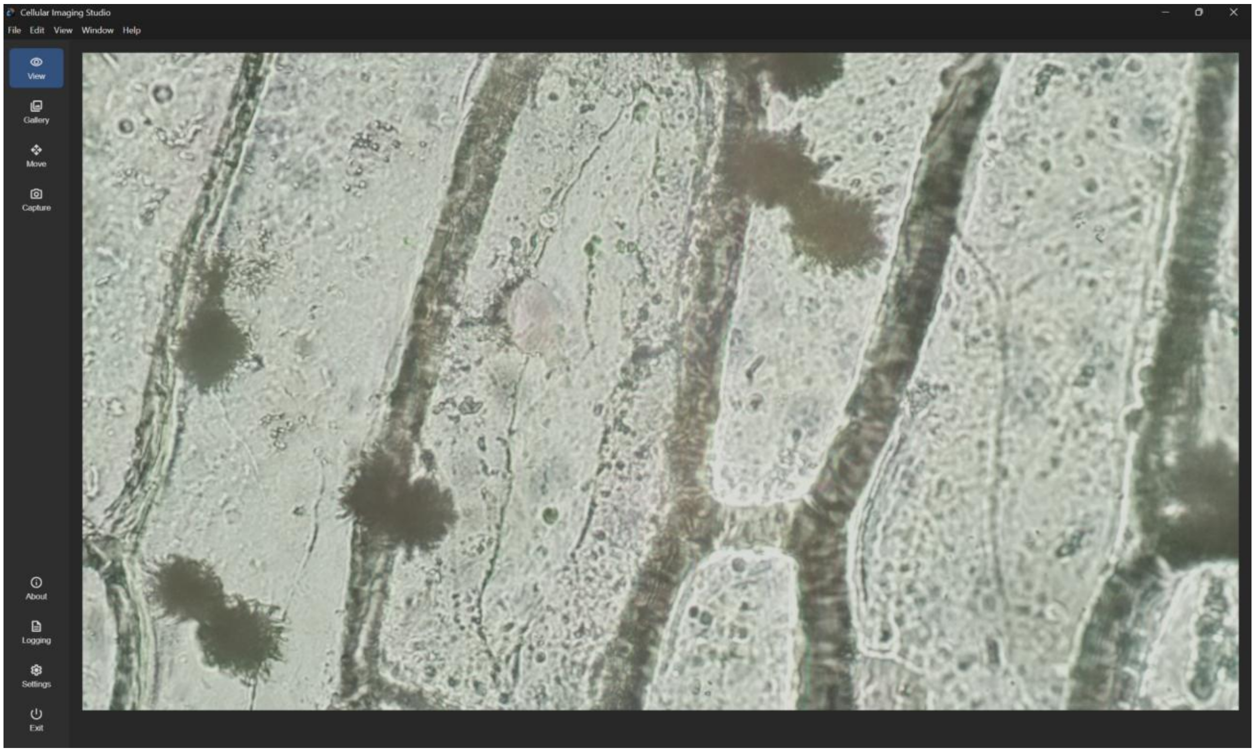
After powering up the microscope, launch the Cellular Imaging Studio program and click the **Connect** button when the start-up screen appears. After successful connection, you'll arrive at the View module.

If no microscope is detected, you'll receive an error message. See the Troubleshooting section of this manual for additional information about connection issues.

Cellular Imaging Studio Walk-Through

This section provides an overview of all the program modules and settings you'll need to capture images and videos, perform AI-driven analyses, export analysis findings, and customize the app to your liking.

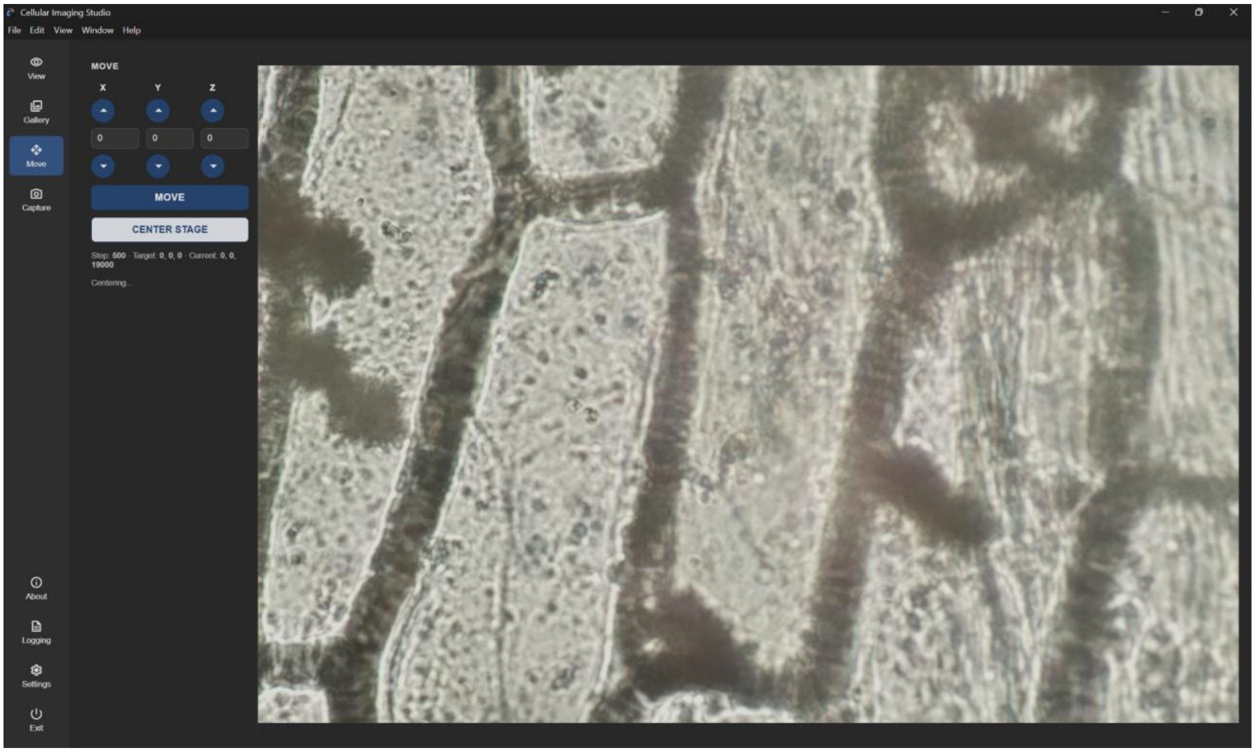
View Module



The View module is the initial app landing page that provides a live preview of the sample slide loaded into the microscope. This module – and all others – includes a sidebar navigation menu to access all the program modules and features.

If the live feed fails to appear, refer to the Troubleshooting section.

Move Module

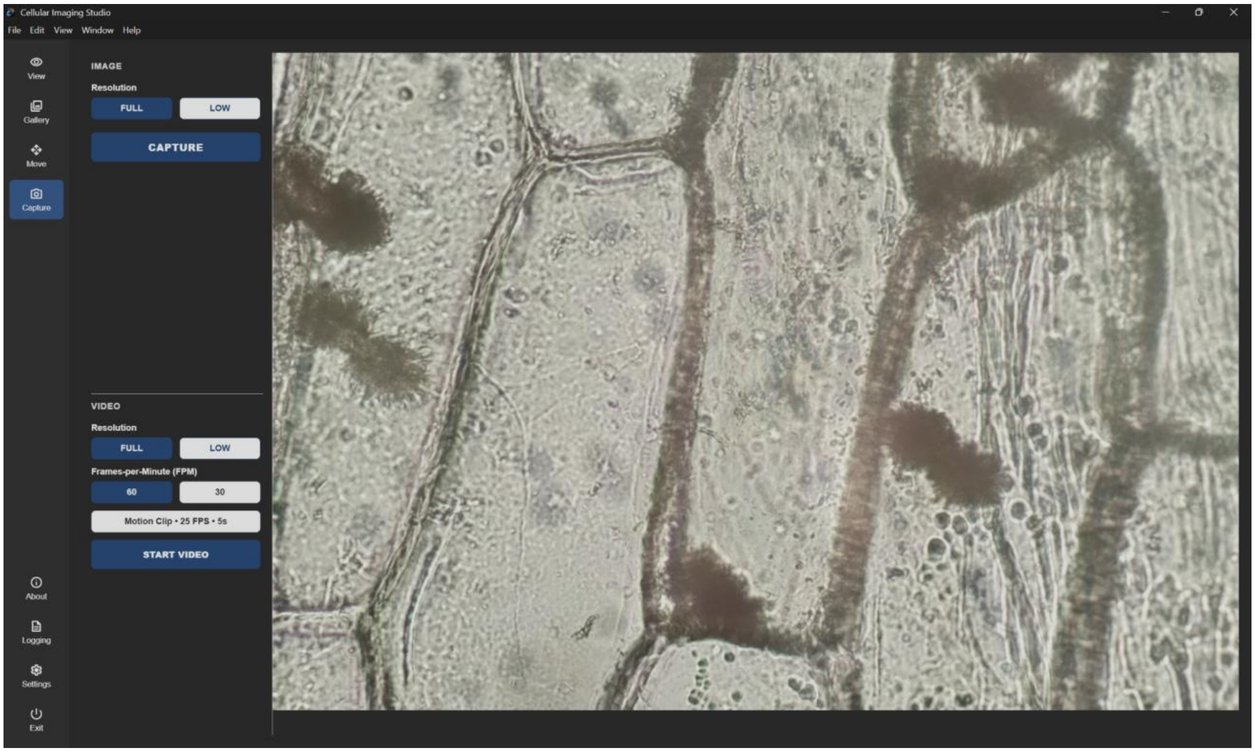


The Move module allows you to reposition the slide, using the microscope's built-in motors to shift the slide by a quantity of unit steps for each axis.

The Z-axis represents the vertical distance between the sample slide and camera, in step increments of approximately 50 nanometers. The X and Y-axis are lateral adjustments, in step increments of approximately 70 nanometers.

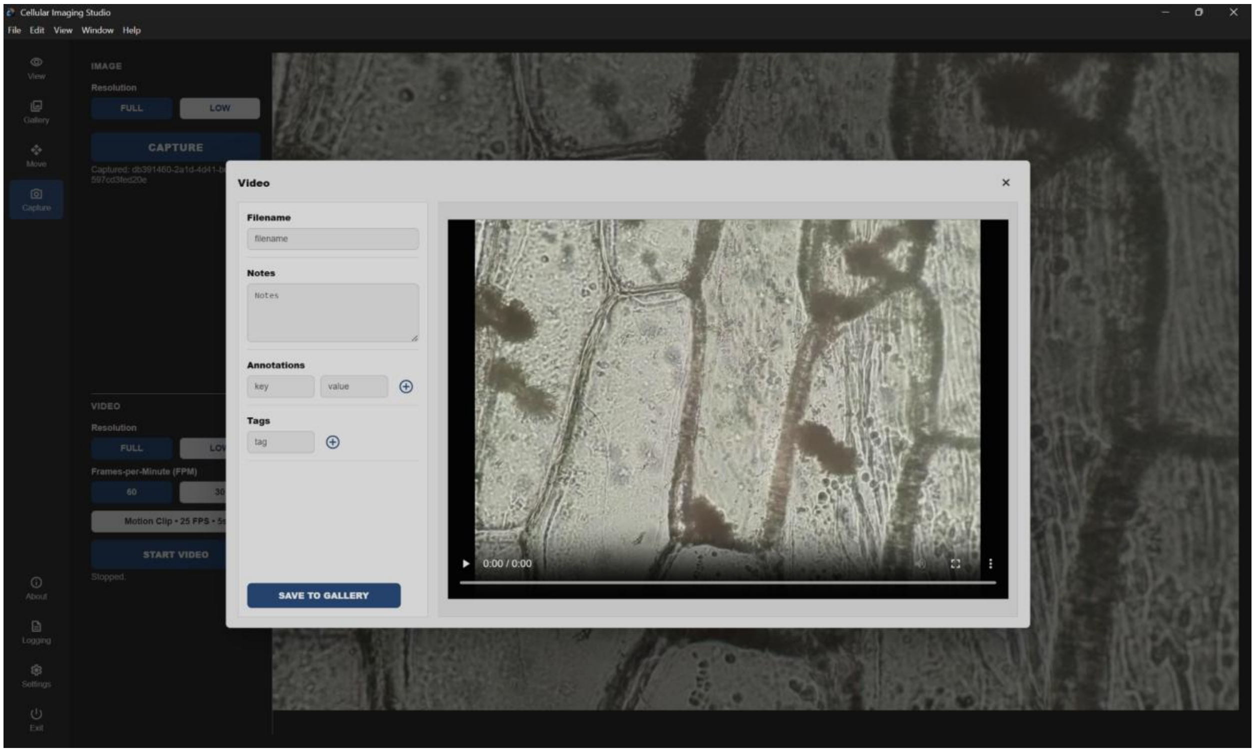
The **Center Stage** button returns the microscope to its default position of (0, 0, 0).

Capture Module



The Capture module provides you with options to capture either a still image or video clip, including resolution options for both. The video options also include setting for video duration, frame rate, and taking a time-lapse video.

Capturing an image or video is required prior to performing AI analyses.

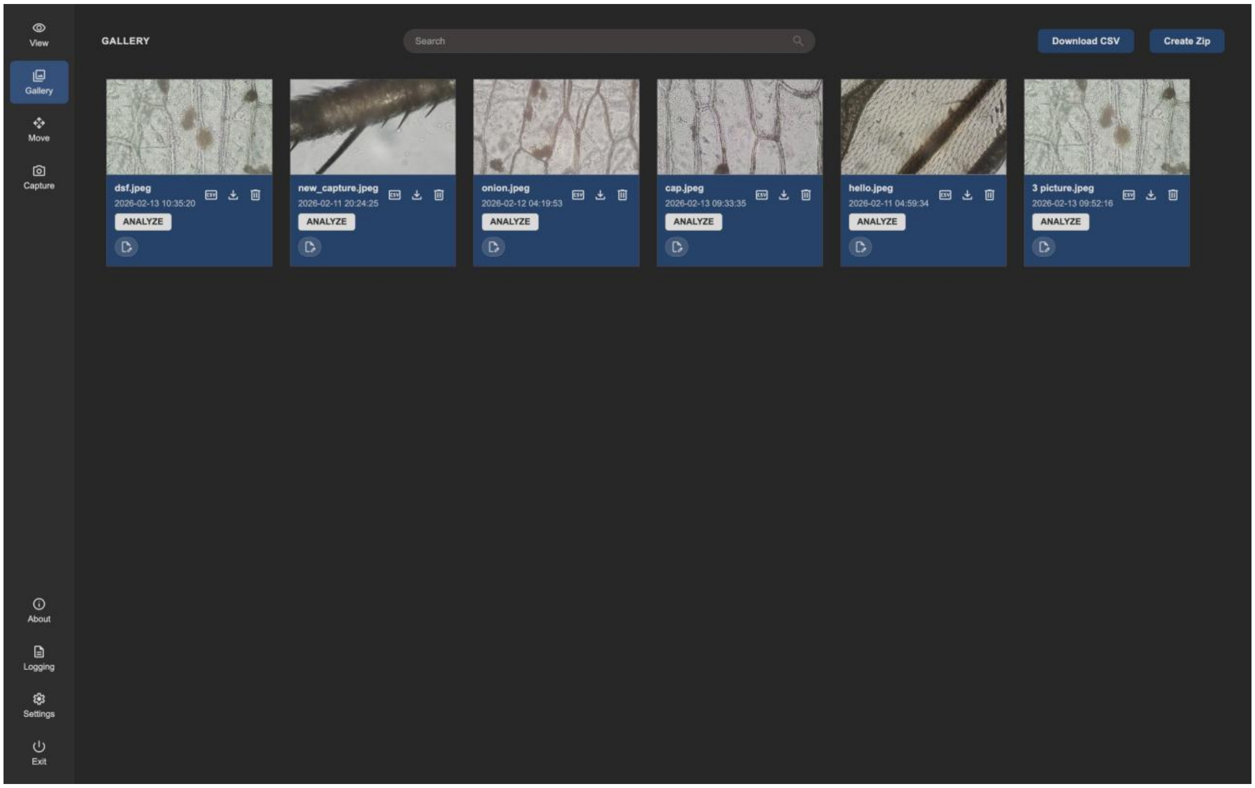


After capturing and image or recording a video, a pop-up appears, containing a preview of the captured media and options for saving it to the gallery, editing the filename, and adding any of the following optional metadata.

- Notes
- Annotations: Key + Value
- Tags

Updating your file metadata is also possible in the Gallery module, but **adding a custom filename is only available within this module**.

Gallery Module



The Gallery module is a collection of all captured images and videos with options to view each file and update their metadata – along with several download options and a search bar to find files by name. The Gallery module is where you initiate our custom AI analyses.

- **ANALYZE** initiates our AI cell counting and motion detection analyses.
- **Create Zip** downloads all videos and images into a single ZIP file.
- **Download CSV** exports a CSV file with a list of metadata of all videos and images.
- **Analysis Download** exports the analysis findings as a CSV file.
- **Media Download** exports a file as a JPEG image or MP4 video.
- **Delete** permanently removes a file.
- **Edit Metadata** allows you to add/modify notes, annotations, and tags.

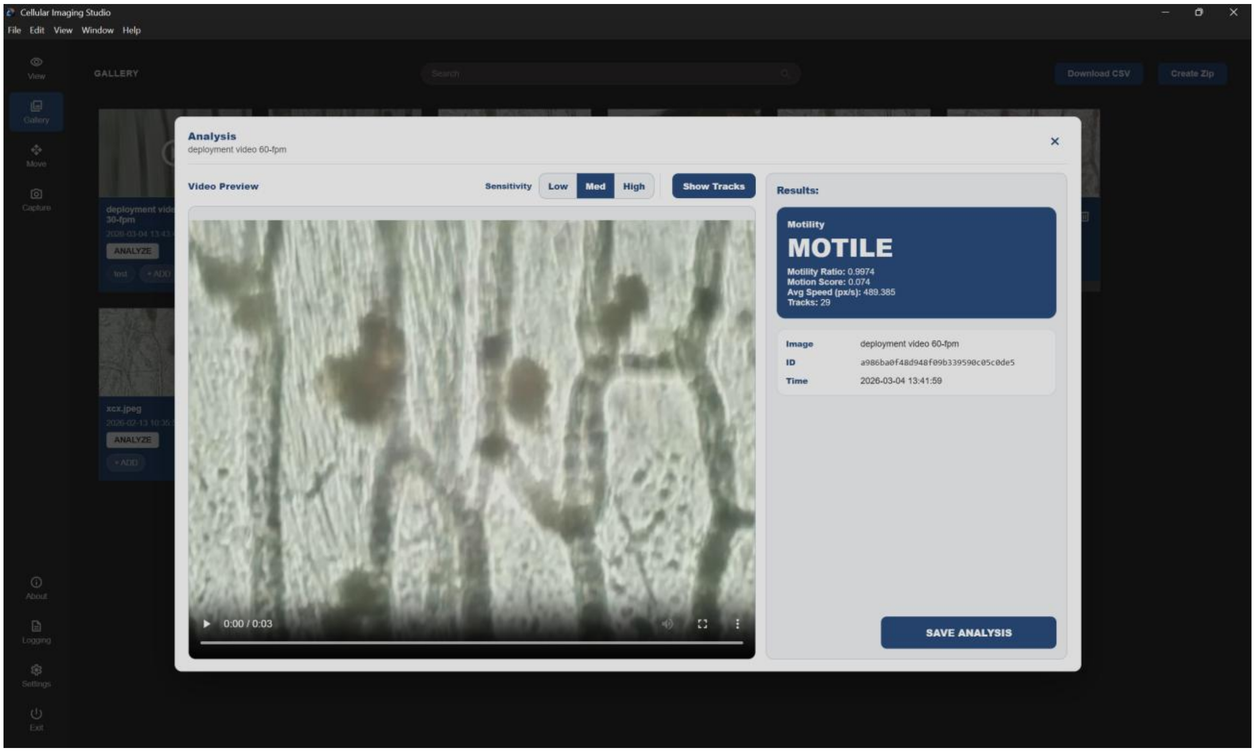
Image Analysis

Clicking the **ANALYZE** button for an image returns the number of objects in the image with an option to save the analysis data.

	A	B
1	ML_BlobCount	ML_AnalyzedAt
2	1062	2026-03-04T21:53:21.143Z

The image analysis data, from the CSV download option from here or the Gallery (as shown in MS Excel), includes the number of cells detected in the image and an analysis timestamp.

Video Analysis



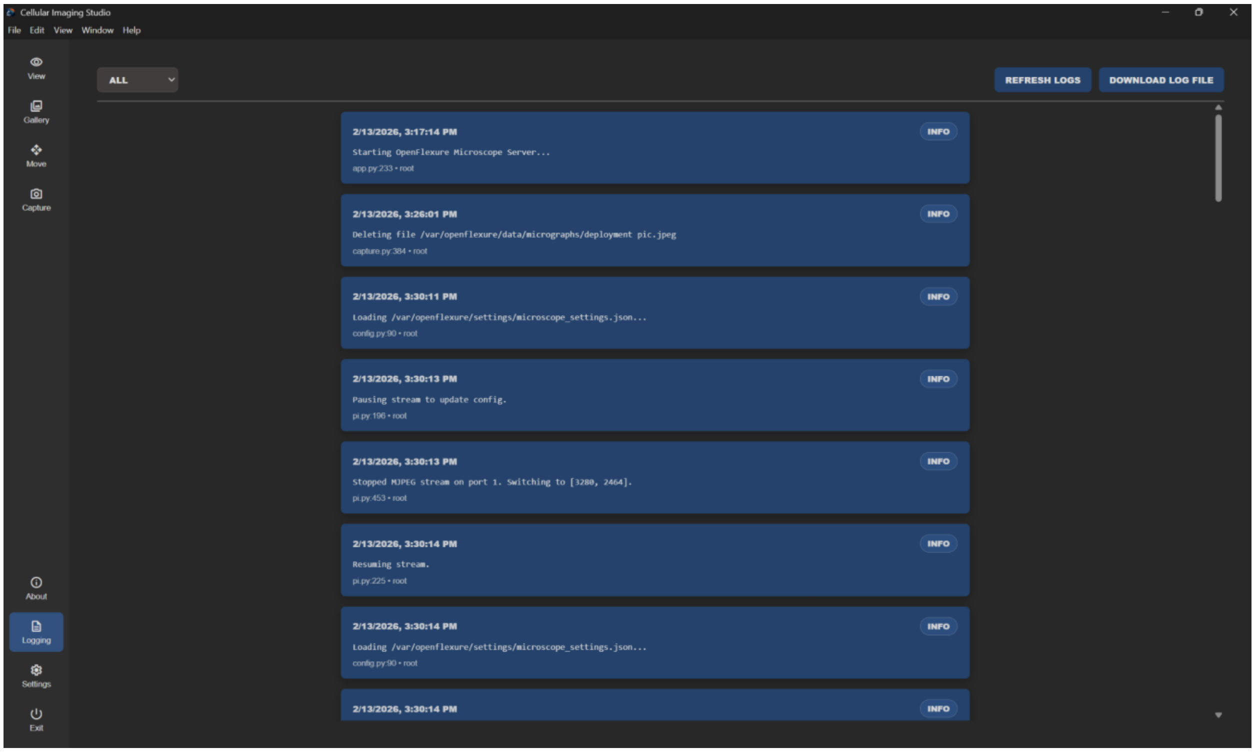
Clicking the **ANALYZE** button for a video returns information about the number of objects and their movements over the course of the video, with an option to save this data too.

	A	B	C	D	E	F	G
1	fpm	Notes	ML_MotilityRatio	ML_MotionScore	ML_AvgSpeedPxPerS	ML_Tracks	ML_AnalyzedAt
2	60	6-fpm	0.9974	0.074	489.385	29	2026-03-04T21:54:43.399Z

The video analysis data, from the CSV download option here or from the [Gallery](#) (as shown in MS Excel), includes following information:

- Frames-per-minute (fpm)
- Mobility ratio: Proportion of detected objects exhibiting movement.
- Motion score: Overall activity in the video.
- Average speed of tracked objects, in pixels per second.
- Tracks: Count of independent trajectories detected.

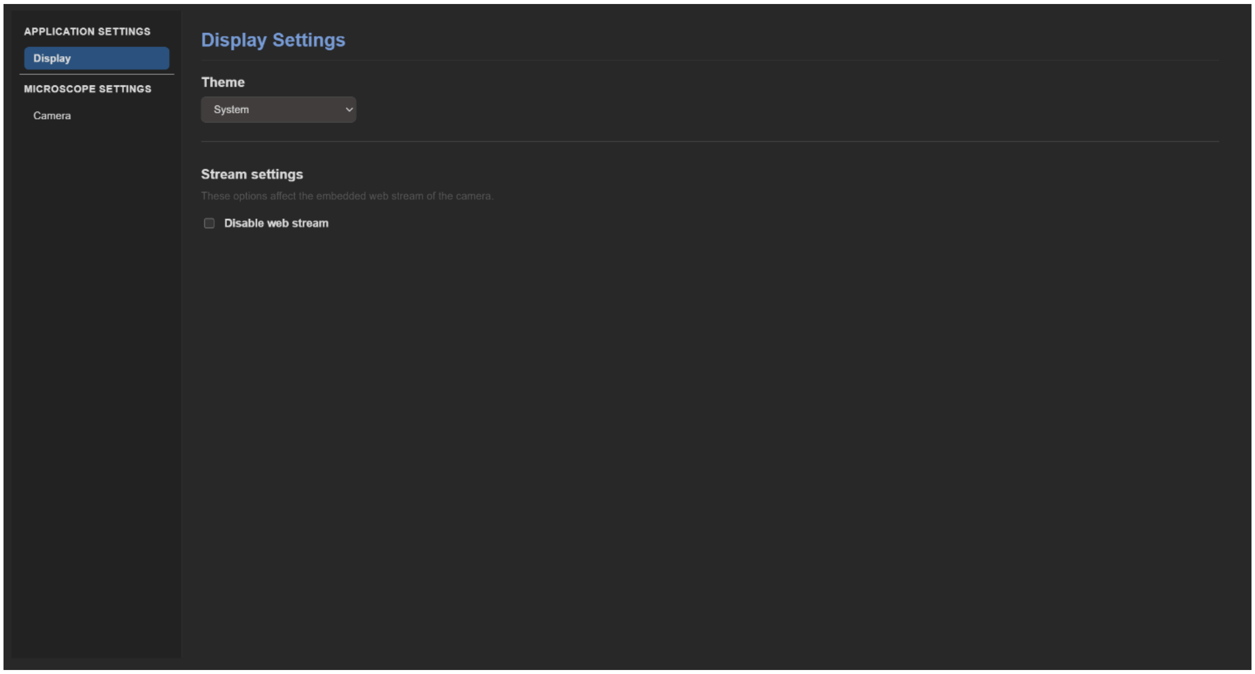
Logging Module



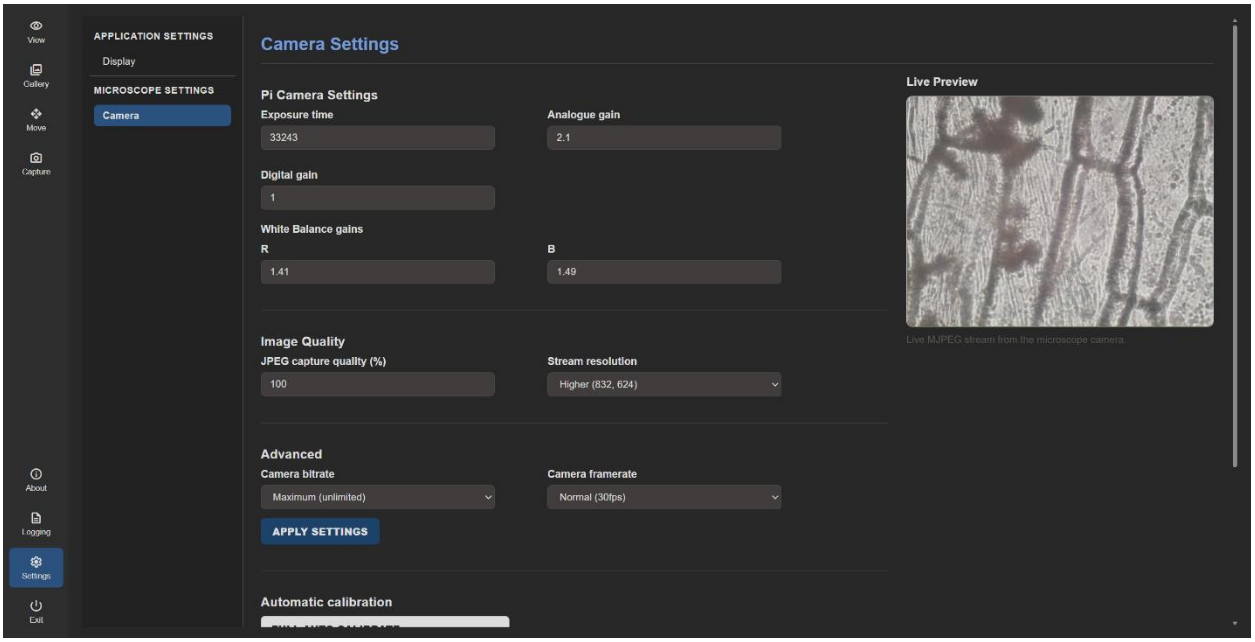
The Logging module provides access to verbose logging of all system events and relative priorities (i.e., ERROR, WARNING, and INFO). This module allows you to refresh the log, filter entries by priority, and download the entire log as a simple text file.

Settings Module

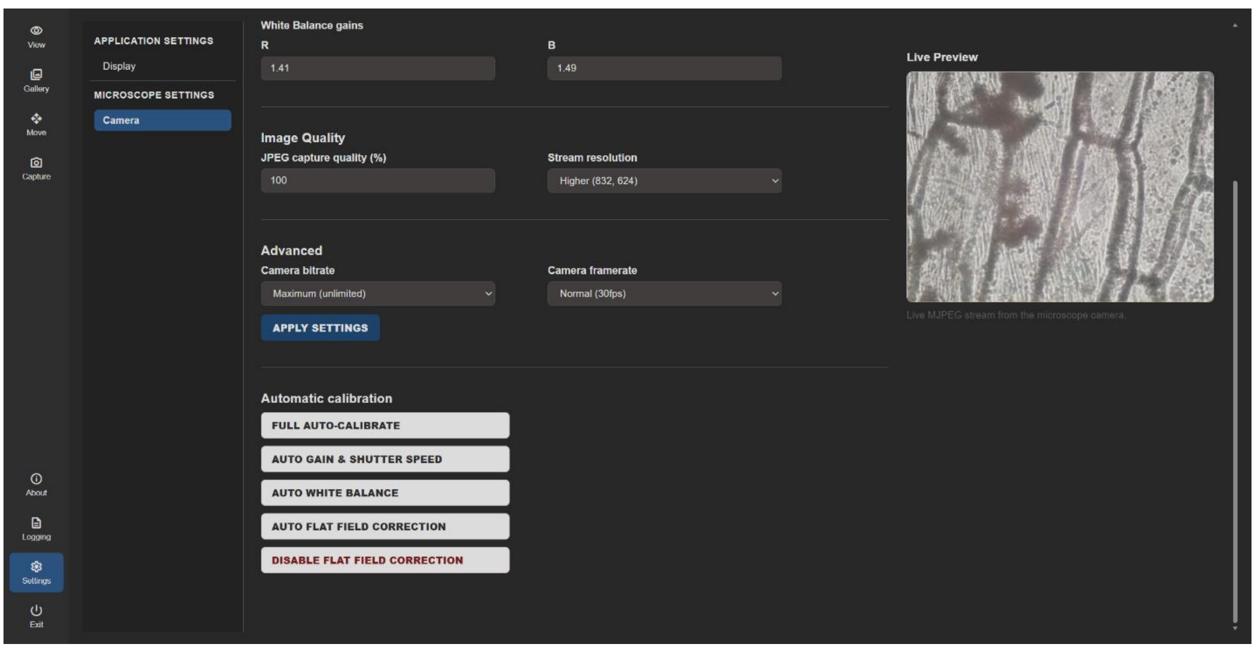
The Settings module includes two submodules, with options to adjust the UI display and update camera settings.



The Display Settings submodule includes options to toggle the UI between dark and light mode and enable/disable the web stream (i.e., the live feed from the camera).

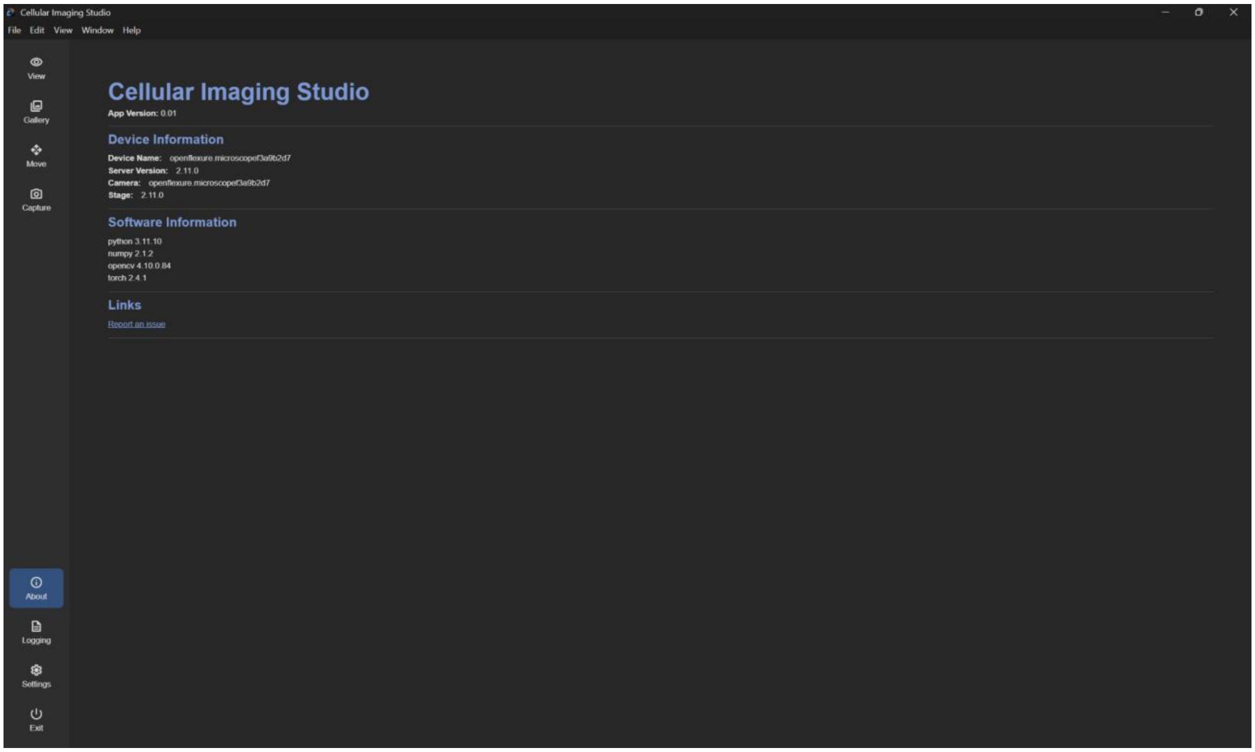


The Camera Settings submodule includes basic color correction settings and data transfer rates for streaming.



The Camera Settings submodule also includes a comprehensive auto-calibration option, automatic exposure and color correction options, and a flat-field correction to resolve imperfections generated by the camera.

About Module



The About module provides useful backend information about the Cellular Imaging Studio app and its dependencies.

This module also includes a link for reporting issues to the development team.

Exit Module

Clicking the Exit button provides you with two options:

- **Shutdown** disconnects the microscope and terminates the program.
- **Restart** also disconnects the microscope but returns you to the initial connection screen instead of exiting the program.

Troubleshooting

Connection Issues

When first booting up the microscope, the Cellular Imaging Studio app might have trouble finding the microscope even when the plug is secure. The microscope might still be booting up, so wait a minute or two before trying to connect again. You can tell when the microscope is ready to connect when the light on the ethernet port starts flashing green.

Building Issues

Some issues might come up when trying to build the app from terminal. Make sure your computer has NPM installed and that you have downloaded ffmpeg into the correct folder:

ml-microscope\vendor\ffmpeg\ffmpeg.exe

Microscope Feed Not Appearing

One of the settings options allows the user to turn off the viewer/live feed from the microscope to check that the box is not selected in Settings>Display>Disable web stream.

If issue remains check your connection to the microscope including internet connection, API connection, and ethernet connection. Internet and API connections can be fixed with a simple reboot of the microscope.

Otherwise, this may be a hardware issue in which case the camera might need to be replaced.

MIT License

Appendix: Software Architecture

Our team used the following software frameworks and libraries to develop the Cellular Imaging Studio app and foster communication between the frontend UI and Python and OpenFlexure servers.

Backend Python Libraries

- CSV: Reading and writing CSV files.
- FastAPI: Frontend UI interaction.
- FFmpeg: Video file handling.
- NumPy: Array and buffer handling for image data.
- OpenCV: Image preprocessing and segmentation for our ML/AI model.
- PathLib: Access resource directories.
- Pydantic: Request models and validation.
- Requests: OpenFlexure server API interaction and custom exception handling.
- Uvicorn: ASGI server for running the backend port.

Frontend Stack

- CSS: Styling module elements.
- Fetch API: Browser-native HTTP client.
- HTML5: Markup language for module layout.
- JavaScript: Core language for UI actions.
- Node.js: JavaScript runtime environment required to run Vite and build the UI.
- npm (node package manager): Frontend dependencies.
- Vite: Dev server used for fast hot-reload development.
- Vue: Reactive JavaScript framework used to build the UI.